

CHATBOT HACKATHON

OVERVIEW & PURPOSE:

The goal of this hackathon is to build a conversational bot to interact with the user and achieve the desired outcome through the conversation. We shall take a two-pronged approach towards this task.

- Use Amazon Alexa APIs to build a bot easily by defining the desired utterances and intents
- See how a bot may be built from scratch and understand the underlying logic

UNDERSTANDING CHATBOTS

Understand the basic definitions of a chatbot and how Alexa Skills help in building them

- What is a Chatbot?
 - [Conversational AI chat-bot — Architecture overview | by Ravindra Kompella | Towards Data Science](#)
- What is Alexa Skills Kit and how does it help with building a chatbot?
 - The Alexa Skills Kit is a software development kit (SDK) that enables a developer to build skills, also called conversational applications, on the Amazon Alexa artificial intelligence assistant

GETTING READY

- Create an **Amazon Developer Account**:
 - To build any type of skill, you need an Amazon developer account. Go to the developer console at <https://developer.amazon.com/>. You can create a new Amazon account. Follow the prompts to enter your registration information and give consent to the Amazon Developer Services Agreement. Once you have done this, you can build skills for Alexa using this account.
- Learn the **Nuances of dialog management in Alexa**:
 - This is critical in building a good, interactive, and intelligent chatbot. Efforts in understanding this will contribute to building a good chatbot. A clear understanding of the different aspects of [Alexa Dialog's management](#) will help to crack the hackathon easily.

TO UNDERSTAND ABOUT AMAZON ALEXA

Go through the below links:

- In this [link](#), you will understand how to build Alexa.
- [Click here](#) to understand how utterances, Intents, and slots are used in Alexa.

CHATBOT ARCHITECTURE

The chatbot is made to be provided as a pluggable service, and configurable through configuration files, in addition to modifying code.

CONCEPTS

Intent: This is a particular skill of a chatbot. It could be a get Zodiac Sign or Movie Suggest, etc. Each intent has a few parameters, which are basically information required to be elicited by the user for the chatbot to complete the particular intent. Example: birth date of the user, language preferences, etc.

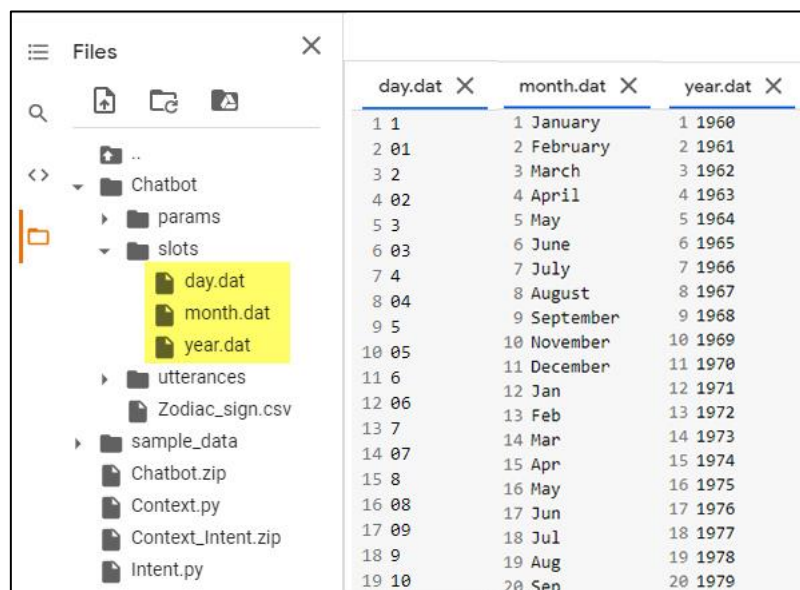
Actions: Each intent also had action that is a function call after all the parameters have been fulfilled.

CONFIGURATION FILES

Slots Folder:

Here each file contains example slots.

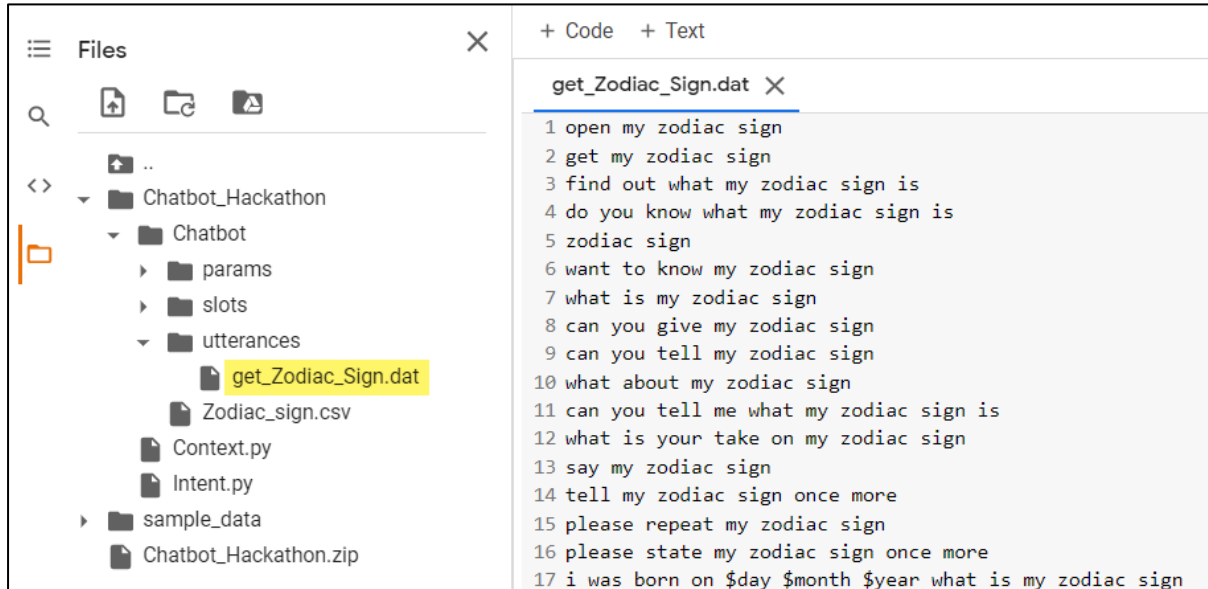
Note: All the slot values are case-sensitive



day.dat	month.dat	year.dat
1 1	1 January	1 1960
2 01	2 February	2 1961
3 2	3 March	3 1962
4 02	4 April	4 1963
5 3	5 May	5 1964
6 03	6 June	6 1965
7 4	7 July	7 1966
8 04	8 August	8 1967
9 5	9 September	9 1968
10 05	10 November	10 1969
11 6	11 December	11 1970
12 06	12 Jan	12 1971
13 7	13 Feb	13 1972
14 07	14 Mar	14 1973
15 8	15 Apr	15 1974
16 08	16 May	16 1975
17 09	17 Jun	17 1976
18 9	18 Jul	18 1977
19 10	19 Aug	19 1978
	20 Sep	20 1979

Utterances Folder:

Here each file contains example utterances for each intent. Note the placeholders starting with \$ for entity masking (as done by get_attributes())



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with a folder named 'Chatbot_Hackathon' containing a sub-folder 'Chatbot'. Inside 'Chatbot', there are folders 'params', 'slots', and 'utterances'. The 'utterances' folder is expanded, showing a file named 'get_Zodiac_Sign.dat' which is highlighted. Other files in the 'utterances' folder are 'Zodiac_sign.csv', 'Context.py', and 'Intent.py'. The code editor on the right shows the content of 'get_Zodiac_Sign.dat', which is a list of 17 example utterances for the 'get_Zodiac_Sign' intent. The utterances are numbered 1 through 17. The last utterance (17) contains placeholders for day, month, and year: 'i was born on \$day \$month \$year what is my zodiac sign'.

```
1 open my zodiac sign
2 get my zodiac sign
3 find out what my zodiac sign is
4 do you know what my zodiac sign is
5 zodiac sign
6 want to know my zodiac sign
7 what is my zodiac sign
8 can you give my zodiac sign
9 can you tell my zodiac sign
10 what about my zodiac sign
11 can you tell me what my zodiac sign is
12 what is your take on my zodiac sign
13 say my zodiac sign
14 tell my zodiac sign once more
15 please repeat my zodiac sign
16 please state my zodiac sign once more
17 i was born on $day $month $year what is my zodiac sign
```

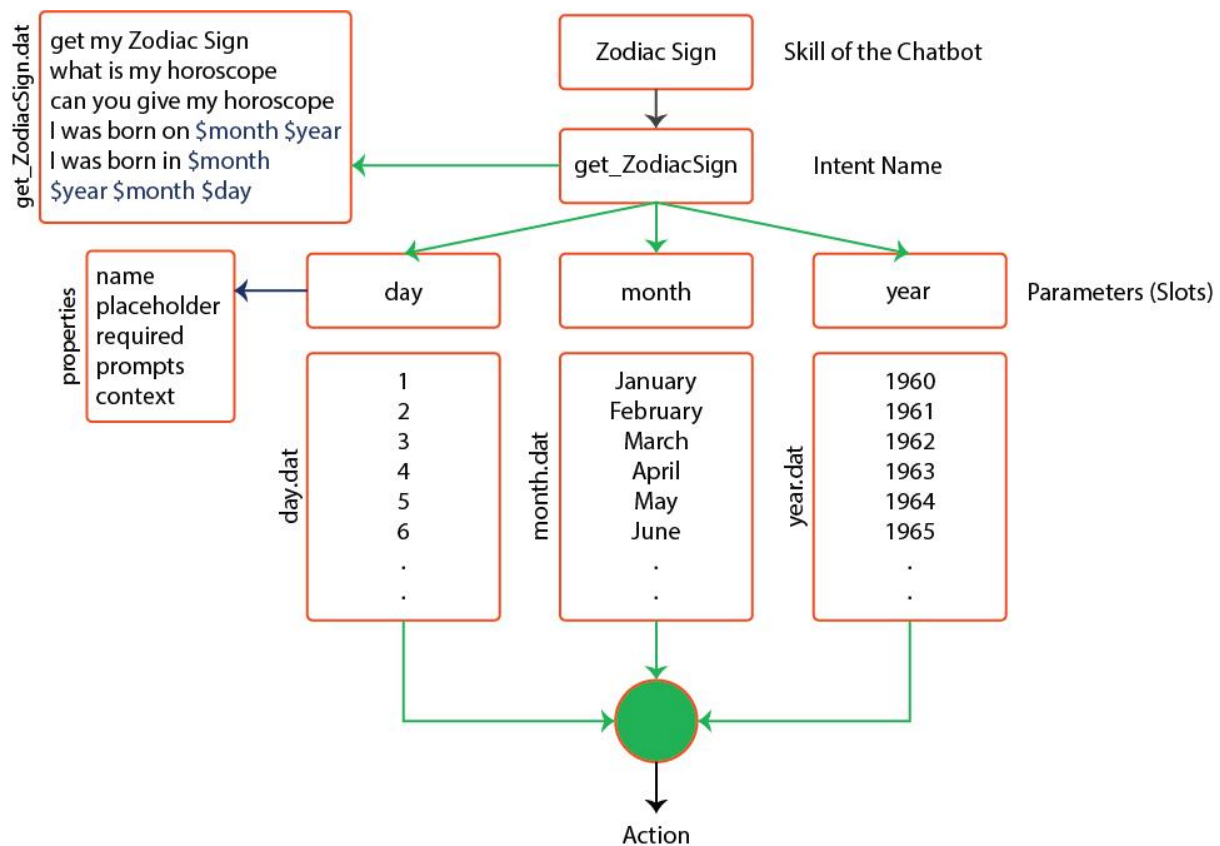
params/params.cfg :

This file contains the JSON which holds the parameters for each intent, and its corresponding values, such as placeholder, required, etc.



The screenshot shows a code editor with a file explorer on the left and a code editor on the right. The file explorer shows a project structure with a folder named 'Chatbot_Hackathon' containing a sub-folder 'Chatbot'. Inside 'Chatbot', there are folders 'params', 'slots', and 'utterances'. The 'params' folder is expanded, showing a file named 'params.cfg' which is highlighted. Other files in the 'params' folder are 'Zodiac_sign.csv', 'Context.py', and 'Intent.py'. The code editor on the right shows the content of 'params.cfg', which is a JSON file defining parameters for the 'get_Zodiac_Sign' intent. The JSON structure includes the intent name, a list of parameters, and a context string. The parameters are defined with their names, placeholders, required status, and prompts. The context string is empty.

```
1 {
2
3   "get_Zodiac_Sign": {
4     "intentname": "get_Zodiac_Sign",
5     "Parameters": [
6
7       {
8         "name": "day",
9         "placeholder": "$day",
10        "required": "True",
11        "prompts": [
12          "Which day(date) were you born?","I was born on 6th July when were you born"
13        ],
14        "context": ""
15      },
16      {
17        "name": "month",
18        "placeholder": "$month",
19        "required": "True",
20        "prompts": [
21          "In Which month you were born?","My birth month is July, what is yours?"
22        ],
23        "context": ""
24      },
25    ]
26  },
27 }
```



Picture depicting the various configuration files pipeline

Class Objects

- **Contexts** (context.py) - Here contexts are some flags that we switched on with each of the parameters or these can be used in conditional if-else statements. Below are the functions:
 - activate_context()
 - deactivate_context()
- Classes which are inheriting the properties of Context
 - FirstGreeting()
 - IntentComplete()
- **Intents** (intent.py) - Every Intent has its name and particular action and parameters. Parameter class is used to hold the attributes of intent from the JSON file.

Conversation Flow:

The image below shows how a given user input affects the chatbot's conversational flow using different methods and returns the appropriate response.

Take the user input from the recursive while loop, Eg: **I was born on 10 January 1996 what is my Zodiac Sign.**

The Reply() function from the Session class takes the user input argument and returns the respective prompt.

The following are the four main variables that flow through the process:

1. The user given input: **user_input = I was born on 10 January 1996 what is my Zodiac Sign**
2. Context from (context.py): context.name registers either "FirstGreeting" or "IntentComplete" as input, **context = FirstGreeting**
3. Attributes: A dictionary which stores the respective filled slots information initially Empty. i.e., **attributes = {}**
4. Current_intent: The intent predicted by the model based on the user input, initially None. i.e., **current_intent = None**

input_processor() helps in identifying all the slots in the user utterance. Identify and map them to the parameters.

attributes = {'day': '10', 'month': 'January', 'year': '1996'}

clean_input = "i was born on \$day \$month \$year what is my zodiac sign"

The model generates a score for intent identification based on stored sample intent utterances upon cleaned user data. Following that, a logical decision making based on contexts at intent identifier to switch to a new intent or remain at the same can be done. loadIntent() takes the path (Chatbot/params/params.cfg) and intent_name (get_ZodiacSign) as inputs and Return's object of Intent class (intent.py) with updated name; parameters; actions of current Intent's.

current_intent = object of get_ZodiacSign intent

check_required_params() goes through required parameters of an intent object and generates a reply for user to provide that missing parameter. The prompt will be None if all the parameters are satisfied

prompt = None; context = FirstGreeting

If prompt is None and the context is still FirstGreeting the check_actions() function assigns the context.name value to "IntentComplete" and performs the action for the intent as mentioned in the intent config file.

The check_action() function performs actions pertaining to current intent value and returns the appropriate prompt and context values

prompt = Your Zodiac sign is Capricorn

context = IntentComplete

attributes = {'day': '10', 'month': 'January', 'year': '1996'}

